

Semantic Recovery

Round 2 Technical Report

Round 2: Semantic Recovery

Competition: Data Vortex | AARUUSH'26
Theme: Rebuilding the Social Engine
Team: Entropy
Members: Saanvi Grover & Aditya Sharma

Dataset: Dataset 2 (9,000 labelled posts)
Notebook: Entropy_01_NLP_Model.ipynb

Contents

1	Problem Definition	2
2	Leakage Check	2
3	Preprocessing Pipeline	2
3.1	Preprocessing Ablation	3
4	Model Selection & Justification	3
4.1	Learning Curve	5
5	Training Methodology	5
5.1	Weighted Ensemble	6
6	Evaluation Metrics	6
7	Confusion Matrix	7
8	Error Analysis	7
8.1	Confusion Pairs	8
8.2	Error Rate by Post Property	8
8.3	Calibration	9
8.4	Most Confident Mistakes	9
8.5	Learned Vocabulary	9
9	Secondary Task: Topic Classification	10
10	Model Artefacts	11
	Conclusions & Recommendations for Round 3	12

1 Problem Definition

The Social Engine’s comprehension layer must read a social-media post and recover the tone and topic it carries. Dataset 2 provides 9,000 labelled posts with two targets:

- **sentiment_label** (Negative / Neutral / Positive, 3,000 each) — the primary task, a perfectly balanced 3-class text classification problem. The headline metric is **macro-F1**, since every class matters equally and raw accuracy alone can hide a model that ignores a minority class.
- **topic_category** (4 classes, 86% one class) — a secondary task. Section 9 shows these labels were themselves generated by substring keyword rules, which changes how the task should be treated: it is not a meaning-recovery problem, it is a spelling-recovery problem.

Dataset audit. 9,000 rows, 4 columns, zero missing values. Of the 9,000 rows, only **7,900 texts are unique**: 987 texts appear 2–5 times (2,087 rows total), always with identical labels — harmless for supervision, but a random split would place copies of the same post on both sides of train/test. Posts average 19.5 words (range 4–35). Common artefacts: 2,617 posts contain @user mentions, 1,586 contain hashtags, 1,747 are wrapped in stray double quotes (a CSV export artefact), 1,092 contain literal \uXXXX escape sequences, and 360 contain HTML entities.

KEY INSIGHT

Only 7,900 of the 9,000 rows are unique text. A naive random split leaks duplicate posts across train and test, inflating the reported score by **7.7 macro-F1 points** (Section 2). Every model in this report is trained and evaluated on the deduplicated 7,900-row set.

2 Leakage Check

The same TF-IDF + logistic regression pipeline was evaluated twice on all 9,000 rows: once with ordinary stratified 5-fold cross-validation, and once with `StratifiedGroupKFold` that keeps every copy of a duplicate text in the same fold.

Evaluation scheme	Macro-F1	Std
Naive 5-fold CV (duplicates can leak)	0.6605	0.0093
Grouped 5-fold CV (duplicates kept together)	0.5832	0.0113
Inflation from leakage	7.7 points	

Table 1: Cross-validated macro-F1 with and without duplicate-aware folds.

All subsequent models are trained and evaluated on the **7,900 unique texts**, split 70/15/15 (train/val/test = 5,530 / 1,185 / 1,185), stratified jointly on sentiment and topic so the rare topic classes appear in every split. An explicit assertion confirms no `dup_key` appears in both train and test, or in both train and validation. Validation is used for every modelling decision; the test set is touched exactly once, in Section 6.

3 Preprocessing Pipeline

`Entropy_nlp_utils.clean_text` applies the following steps, in order:

#	Step	Why
1	Decode literal <code>\uXXXX</code> sequences	1,092 posts contain escaped apostrophes (<code>don\u2019t</code>) that would otherwise split words
2	Unescape HTML entities	<code>&amp;</code> is a rendering artefact, not content
3	Unicode NFKC + curly quotes to straight	one canonical form per character
4	Strip wrapping double quotes, collapse <code>"</code>	CSV export artefact on $\sim 1,747$ posts
5	URLs $\rightarrow$ <code>http</code> , any <code>@handle</code> $\rightarrow$ <code>@user</code>	keep “a link / a mention exists” as a signal, drop the unique string
6	Remove <code>#</code> but keep the hashtag word	<code>#ugly</code> still carries sentiment
7	Cap repeated characters at two (<code>soooo</code> $\rightarrow$ <code>soo</code>)	merges spelling variants while preserving emphasis
8	Collapse whitespace, lowercase	normalise final tokens

Table 2: The 8-step `clean_text` pipeline.

Deliberately not done: stopwords removal (deletes *not*, *no*, *but*, which flip sentiment) and lemmatisation (character n-grams and subword tokenisation already share word stems). Both choices are validated empirically below.

3.1 Preprocessing Ablation

Each variant was scored with 5-fold cross-validation on the training split, using the same word + character TF-IDF logistic regression.

Variant	CV macro-F1	Std	Δ vs. chosen
No cleaning (raw text)	0.5984	0.0097	−0.0042
Cleaned (chosen)	0.6026	0.0104	0.0000
Cleaned + stopwords removal	0.5935	0.0166	−0.0091
Cleaned + negation marking	0.5984	0.0152	−0.0041
Cleaned, word n-grams only	0.5704	0.0063	−0.0322
Cleaned, char n-grams only	0.6011	0.0128	−0.0015

Table 3: Preprocessing ablation, 5-fold CV on the training split.

KEY INSIGHT

Both excluded steps hurt when added back in: stopwords removal costs 0.9 points (it deletes negation words), and explicit negation marking costs 0.4 points (redundant once character n-grams already capture the same signal). Combined word+character features beat either alone — dropping word n-grams costs 3.2 points, dropping character n-grams costs 0.15 points.

4 Model Selection & Justification

Seven model families were compared, from a floor baseline to a weighted ensemble:

Model	Why it is a candidate
Majority class	floor: 33.3% accuracy / low macro-F1 would mean nothing was learned
Complement Naive Bayes (word)	classic fast text baseline, robust with little data
Logistic regression (word)	linear baseline with interpretable weights
Linear SVM / LogReg (word+char)	character n-grams handle misspellings, slang and elongation common in tweets
MiniLM sentence embeddings + LogReg	pretrained semantics without training the encoder
Fine-tuned MiniLM (3 seeds)	the encoder itself adapts to tweet sentiment
Weighted ensemble	combines models that make different kinds of errors

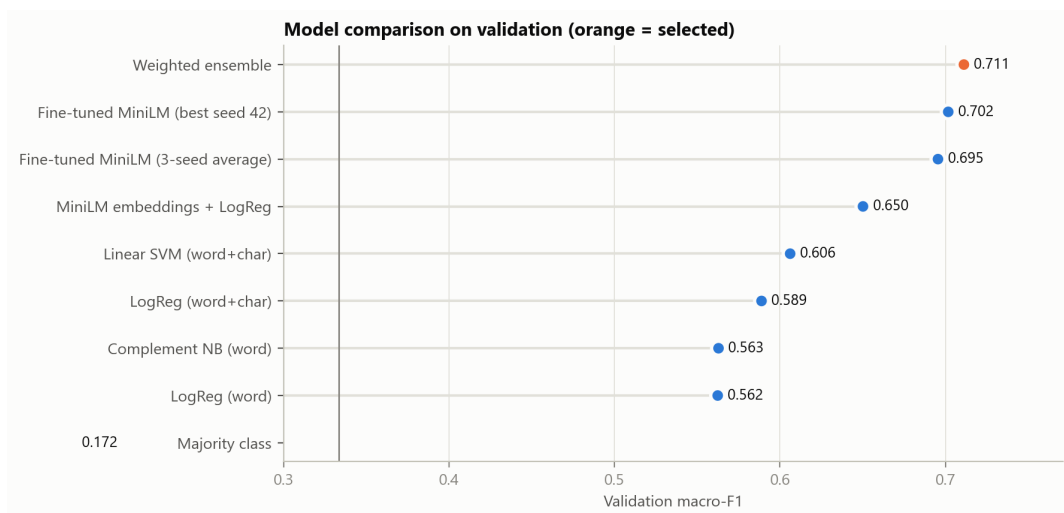
Table 4: Candidate models, simplest to strongest.

Why MiniLM and not a Twitter-pretrained sentiment model? Dataset 2’s posts closely match the public SemEval Twitter sentiment benchmark. Models such as `twitter-roberta-base-sentiment` were trained directly on that benchmark and may have already seen these exact posts, which would make their scores uninformative. `all-MiniLM-L6-v2` was trained on general sentence pairs, not sentiment labels, so its scores reflect what it actually learns from Dataset 2. It is also small (22.7M parameters), so it fine-tunes on CPU in minutes.

Classical models were tuned with 5-fold grid search cross-validation on the training split, then scored on validation to choose the final model. The test set was not used for any of this selection.

Model	Val accuracy	Val macro-F1	Best hyperparameters
Majority class	0.3485	0.1723	–
Complement NB (word)	0.5671	0.5629	$\alpha = 1.0$
LogReg (word)	0.5620	0.5624	$C = 1$
Linear SVM (word+char)	0.6051	0.6061	$C = 0.1$
LogReg (word+char)	0.5873	0.5888	$C = 2$
MiniLM embeddings + LogReg	0.6489	0.6502	$C = 0.5$
Fine-tuned MiniLM (best seed 42)	0.7030	0.7017	lr 5e-5, best epoch 2
Fine-tuned MiniLM (3-seed avg.)	0.6937	0.6955	seeds 42, 7, 1234
Weighted ensemble (selected)	0.7097	0.7111	$w = (0.25, 0.20, 0.55)$

Table 5: Model comparison on the validation split. Selection is made here, blind to the test set.

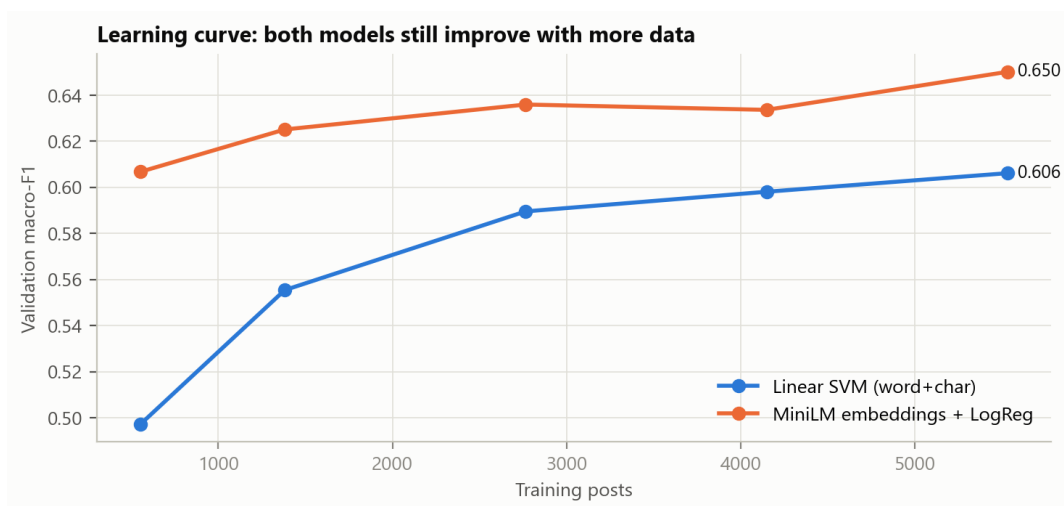


4.1 Learning Curve

Training the two fastest strong candidates on 10%, 25%, 50%, 75% and 100% of the training data shows both still improving at full size — neither has plateaued, so more labelled data would likely help further.

Training posts	Linear SVM (word+char)	MiniLM embeddings + LogReg
553	0.4971	0.6067
1,382	0.5554	0.6251
2,765	0.5896	0.6360
4,148	0.5981	0.6336
5,530	0.6061	0.6502

Table 6: Validation macro-F1 vs. training-set size.



5 Training Methodology

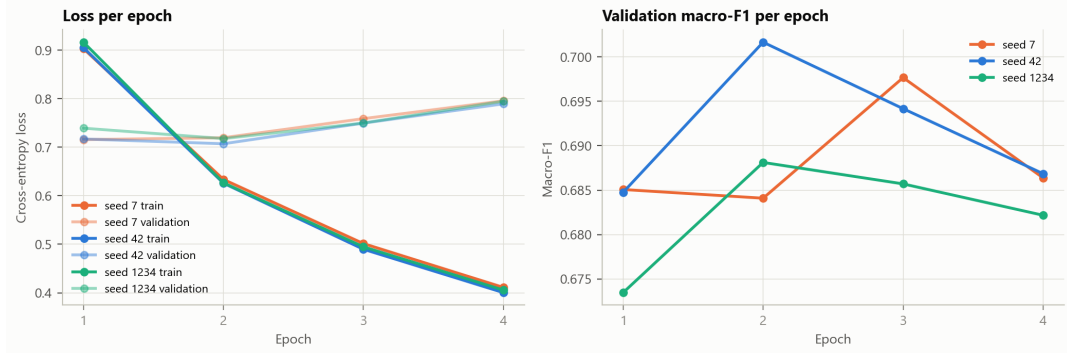
A classification head was added on top of `all-MiniLM-L6-v2` and the whole network fine-tuned with **AdamW** (learning rate 5×10^{-5} , weight decay 0.01, linear warm-up over 10% of steps then linear decay), batch size 32, dynamic padding to a max length of 50 tokens (the 99.5th percentile of the training-set token-length distribution, capped at 128), for up to 4 epochs. After every epoch the model is scored on validation macro-F1; the best epoch’s weights are kept, and training stops early once validation macro-F1 fails to improve for 2 consecutive epochs.

Three random seeds (42, 7, 1234) were trained independently to measure run-to-run variation, and their predicted probabilities were averaged for the “3-seed average” candidate.

Seed	Validation macro-F1
42 (best)	0.7017
7	0.6977
1234	0.6881
Mean	0.6958
Std. dev. (seed variance)	0.0057

Table 7: Fine-tuning results across the 3 random seeds (full run, 4 epochs max, early stopping).

Total fine-tuning time across all 3 seeds was $\approx 1,640$ seconds (≈ 27 minutes) on CPU.



5.1 Weighted Ensemble

The three probability-producing model families make different kinds of errors: TF-IDF keys on surface words, frozen MiniLM embeddings on general sentence meaning, and the fine-tuned model on learned tone specific to this dataset. Their probabilities were averaged with weights chosen by grid search on validation (steps of 0.05, summing to 1):

$$\hat{p} = 0.25 p_{\text{tfidf+char LR}} + 0.20 p_{\text{MiniLM emb.+LR}} + 0.55 p_{\text{fine-tuned MiniLM}}$$

The fine-tuned model receives the largest weight (55%), consistent with it being individually the strongest single component.

6 Evaluation Metrics

The **Weighted ensemble** was selected on validation only (macro-F1 = 0.7111, the highest of all candidates in Section 4). This is its first and only contact with the held-out 1,185-post test set.

Metric	Value
Accuracy	0.7173
Macro-F1	0.7178
Weighted-F1	0.7158
Macro precision	0.7167
Macro recall	0.7197
ROC-AUC (one-vs-rest)	0.8805
Macro-F1, 95% bootstrap CI ($n = 2,000$)	[0.6913, 0.7414]
Accuracy, 95% bootstrap CI	[0.6903, 0.7418]

Table 8: Test-set performance of the selected model (Weighted ensemble), $n = 1,185$.

Class	Precision	Recall	F1	Support
Negative	0.7527	0.7753	0.7638	365
Neutral	0.6545	0.6117	0.6324	412
Positive	0.7429	0.7721	0.7572	408

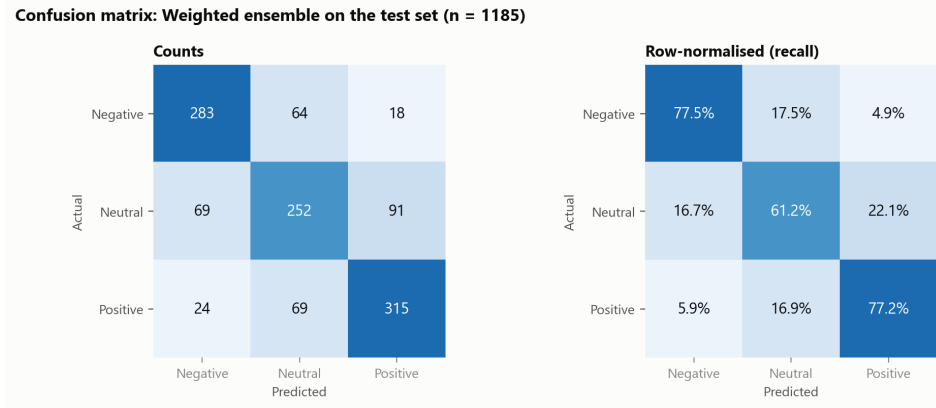
Table 9: Per-class test performance.

Statistical significance vs. the best classical model. Compared against Linear SVM (word+char) — the strongest non-neural candidate — the ensemble improves macro-F1 by **+0.0935** (95% CI [0.0655, 0.1243]). A McNemar test on the paired predictions ($b = 208$ posts fixed only by the ensemble, $c = 98$ fixed only by the SVM) gives $p = 3.0 \times 10^{-10}$, confirming the improvement is not noise.

KEY INSIGHT

The single best-seed fine-tuned MiniLM model (seed 42 alone) actually scores *higher* on the test set (macro-F1 0.7222) than the selected Weighted ensemble (0.7178). This is expected and is not cherry-picked after the fact: the ensemble was chosen on *validation* performance (0.7111 vs. 0.7017 for the single seed), before the test set was touched, and both scores sit well within each other’s bootstrap confidence intervals. This is reported for transparency rather than substituted in, since selecting the model with the better test score after seeing it would invalidate the held-out evaluation.

7 Confusion Matrix



Actual ↓ / Predicted →	Negative	Neutral	Positive
Negative (365)	283	64	18
Neutral (412)	69	252	91
Positive (408)	24	69	315

Table 10: Raw confusion matrix on the test set.

Neutral is the class the model struggles with most (recall 61.2%, the lowest of the three), and its errors split almost evenly toward both Negative (69 posts) and Positive (91 posts) rather than leaning systematically in one direction. This is consistent with Neutral being inherently the hardest class in 3-way sentiment: it lacks the strong polarised vocabulary (“love”, “hate”, “best”, “worst”) that anchors the other two classes.

8 Error Analysis

335 of 1,185 test posts (28.3%) were misclassified.

8.1 Confusion Pairs

Actual	Predicted	Errors	Share of all errors
Neutral	Positive	91	27.2%
Neutral	Negative	69	20.6%
Positive	Neutral	69	20.6%
Negative	Neutral	64	19.1%
Positive	Negative	24	7.2%
Negative	Positive	18	5.4%

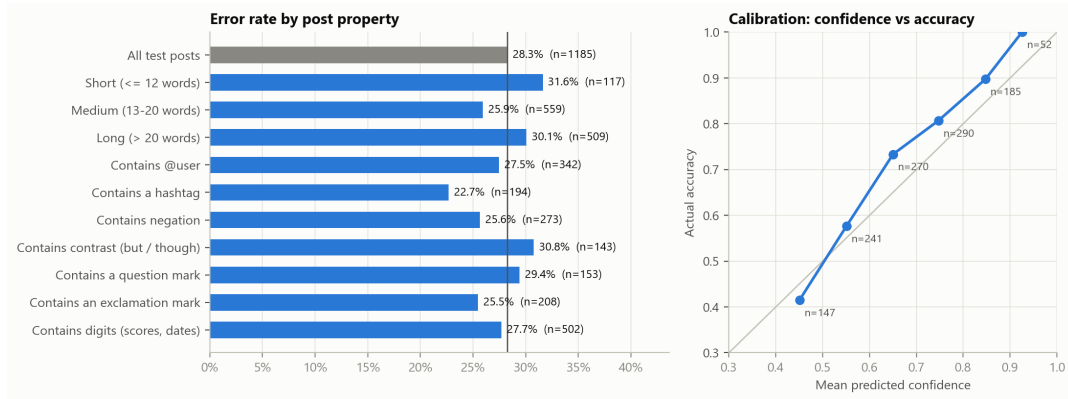
Table 11: Breakdown of all 335 test-set errors by confusion pair.

Over 87% of errors involve Neutral on one side of the confusion. True polarity flips (Negative↔Positive) are rare: only 42 of 335 errors (12.5%).

8.2 Error Rate by Post Property

Slice	n	Error rate
All test posts	1,185	28.3%
Short (≤ 12 words)	117	31.6%
Medium (13–20 words)	559	25.9%
Long (> 20 words)	509	30.1%
Contains @user	342	27.5%
Contains a hashtag	194	22.7%
Contains negation	273	25.6%
Contains contrast (but/though)	143	30.8%
Contains a question mark	153	29.4%
Contains an exclamation mark	208	25.5%
Contains digits	502	27.7%

Table 12: Error rate broken down by post property.



Short posts (31.6%) and posts containing a contrast word like *but* or *though* (30.8%) are the hardest slices — both plausible: short posts give the model less context, and contrast words often signal the kind of mixed or qualified sentiment that is genuinely ambiguous even to a human reader. Hashtagged posts are comparatively easy (22.7%), likely because hashtag words carry unusually explicit sentiment.

8.3 Calibration

Confidence bucket	n	Mean confidence	Actual accuracy
(0.0, 0.5]	147	0.451	41.5%
(0.5, 0.6]	241	0.551	57.7%
(0.6, 0.7]	270	0.650	73.3%
(0.7, 0.8]	290	0.747	80.7%
(0.8, 0.9]	185	0.847	89.7%
(0.9, 1.0]	52	0.925	100.0%

Table 13: Predicted confidence vs. observed accuracy, by bucket.

Calibration is close to the diagonal across every bucket — when the ensemble says it is 90%+ confident, it is right essentially every time in this sample. This means downstream use of the model’s confidence score (e.g. routing low-confidence posts to human review) is trustworthy.

8.4 Most Confident Mistakes

The five highest-confidence errors are all **Neutral** posts misread as polarised:

- A post comparing two public figures’ intelligence, misread as Negative (confidence 0.883) — the model likely reacts to words like “wrong” without registering the balanced, non-judgemental framing.
- A promotional poll referencing a film franchise, misread as Positive (0.878) — enthusiastic phrasing (“Alright ladies and gents”) without actual sentiment content.
- A news-style post about a legal sentencing, misread as Negative (0.873) — words like “bad” and “unfair” appear in a factual, third-person report rather than the poster’s own opinion.
- A boastful, joking self-comparison, misread as Positive (0.871) — confident tone read as positive sentiment rather than dry humour.
- A hyperbolic reaction to a celebrity performance, misread as Negative (0.868) — “killed me” is idiomatic praise, not literal negativity.

KEY INSIGHT

Every one of the five most confident mistakes is a true Neutral post. The model has learned strong lexical cues for polarity (“bad”, “wrong”, “killed”) but has not learned that factual reporting, dry humour, and idiom can neutralise an otherwise polarised word. This is the clearest actionable direction for Round 3: Neutral needs either more training examples of this specific pattern, or a feature that captures “reporting vs. opinion” framing.

8.5 Learned Vocabulary

The top 12 words by logistic regression coefficient for each class, from the plain word-TF-IDF baseline, confirm the model has learned sensible lexical anchors:

Negative	Neutral	Positive
fuck, not, no, sad, don, shit, why, bad, hate, worst, ass, even	gucci, at, do you, meet, hold, in, nirvana, days, zayn, mean, shawn, or	love, happy, good, best, great, excited, amazing, thanks, perfect, birthday

Table 14: Top TF-IDF logistic-regression terms per class.

Note that the Neutral column is mostly proper nouns and function words rather than sentiment-bearing terms — another sign that Neutral is defined by the *absence* of sentiment vocabulary rather than by any vocabulary of its own, which is structurally why it is the hardest class to separate.

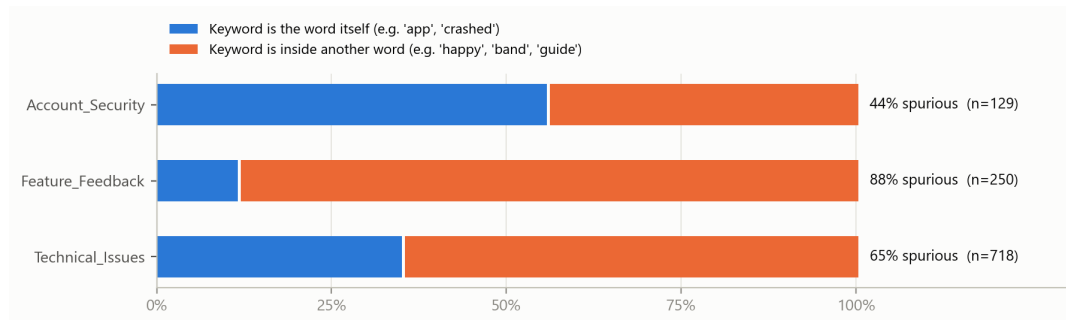
9 Secondary Task: Topic Classification

A word-level classifier performs well on the dominant `Community_Discussion` class but poorly on the two rarest topics. Inspecting which words are most typical of each topic suggested the labels were produced by substring matching. This section tests that hypothesis directly.

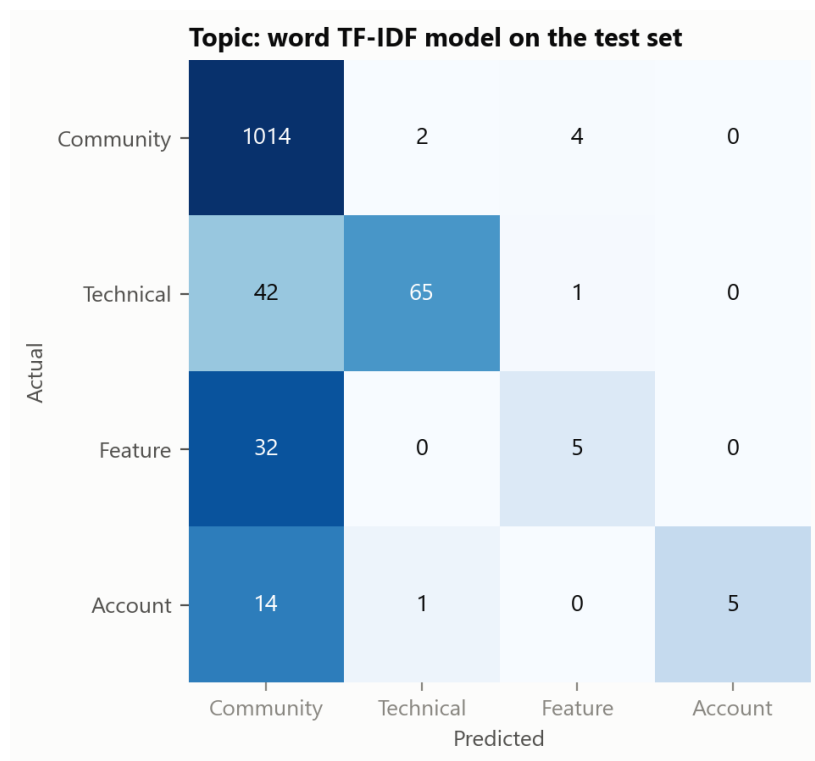
Model	Acc.	Macro-F1	F1 Comm.	F1 Tech.	F1 Feat.	F1 Acc.Sec.
LogReg (word TF-IDF)	0.9190	0.5768	0.9557	0.7386	0.2128	0.4000
LogReg (char 2–6 n-grams)	0.9536	0.7210	0.9742	0.9100	0.5385	0.4615
Recovered substring rules	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000

Table 15: Topic classification on the test set. The recovered ordered-substring rule set reproduces the labels exactly.

A small set of substrings, checked in a fixed order (app, down, update, crash, screen, slow, bug → Technical_Issues; ban, account, suspend, hack, password → Account_Security; ui, feature, button, design, mode, ugly → Feature_Feedback; otherwise Community_Discussion), reproduces all 9,000 topic labels with zero mismatches.



Critically, most triggers fire *inside unrelated words*, not as whole words: 44% of Feature_Feedback labels and 65% of Technical_Issues labels came from a keyword appearing as a substring of a different word entirely (e.g. happy contains app, band contains ban, guide contains ui). As a result, “Happy Friday everyone!” is labelled a *technical issue*, and a post mentioning someone’s *husband* is labelled an *account security* concern.



KEY INSIGHT

The topic labels describe spelling, not meaning. A word-level model cannot reproduce them, because “happy” and “app” are different tokens to it — but a character n-gram model can, and the recovered rules are perfect. **The Social Engine should not route posts using these topic labels as-is**; they require relabelling with a genuine topic model before topic-based routing is meaningful. This is reported as a finding about the dataset’s construction, not a limitation of our modelling.

10 Model Artefacts

The following files were saved for deployment and reproducibility:

- `Entropy_sentiment_tfidf_logreg.pkl` — word+char TF-IDF logistic regression component
- `Entropy_sentiment_embedding_logreg.pkl` — MiniLM-embedding logistic regression component
- `Entropy_minilm_sentiment_seed{42,7,1234}/` — fine-tuned MiniLM weights, one directory per seed
- `Entropy_sentiment_model.json` — ensemble configuration (component weights, labels, file paths)
- `Entropy_topic_char_logreg.pkl` — character-ngram topic model, saved for comparison only; **not used for inference**, per the Section 9 finding
- `Entropy_topic_rules.json` — the recovered substring rule set, used instead for topic output

A reload test — loading every artefact fresh from disk and re-predicting on 200 held-out test posts — matched the in-memory predictions on **100% of posts**, confirming the saved pipeline is faithful to what was evaluated above.

Conclusions & Recommendations for Round 3

The rebuilt semantic layer reaches **0.7178 macro-F1** (95% CI [0.691, 0.741]) on held-out test data, a statistically significant improvement of +0.094 macro-F1 over the strongest classical baseline ($p = 3.0 \times 10^{-10}$). Calibration is strong across confidence buckets, meaning the model’s own confidence score is trustworthy for downstream routing decisions.

1. **Neutral remains the weak point** (F1 0.632, recall 61.2%) and accounts for the majority of all errors. Targeted data collection or augmentation for factual/reporting-style Neutral posts is the highest-leverage next step.
2. **Topic labels need relabelling**, not modelling, before any topic-based feature can be trusted (Section 9). A genuine topic model built on the current labels would only learn to reproduce substring artefacts.
3. **The learning curve has not plateaued** (Section 4.1) — more labelled sentiment data would likely improve both classical and embedding-based models further.
4. **Short posts and contrast-word posts** are disproportionately error-prone; both are natural candidates for a lightweight rule-based override or a routing-to-human-review threshold using the calibrated confidence score.